```
class Blog extends controller.Controller
```

The idiomatic way to write `require` in CoffeeScript uses destructuring assignment (see chapter 7) on the object returned, avoiding the repetitiveness of unpacking properties from the controller:

```
# -- blog.coffee --
{Controller} = require './controller'
class Blog extends Controller
```

Destructure the Controller property from the controller module.

Now you have a local variable `Controller` in blog.coffee that references the `Controller` class from controller.coffee.

Camel case for exports

Although variable naming can be a matter of taste, all of the standard libraries for JavaScript, whether in the browser or not, use camel case for naming all functions, methods, and properties. Thus, in the interests of being a good module citizen, you should *always* use camel case for anything you export from a module. So, instead of `exports.myproperty`, it should be `exports.myProperty` or `exports.myClass` if you're exporting a class. In view of this, you might also choose to use camel case for all variables. It's up to you.

In summary, to keep different parts of the application in different files, you need to `export` from one file and `require` in the other. Suppose you have a module_a.coffee file and a module_b.coffee file in the same directory:

module_a.coffee

```
w = 3
x = {a: 1}
```

module_b.coffee

```
z = require('./module_a).y # z == {a: 1}
```